STATE OF THE ART

An Agentic Framework for IaC Misconfiguration Analysis and Remediation

Ahmedou Yahye Kheyri · Supervised by: Hala Bezine

March 2026

PLAN DE L'EXPOSÉ

01 **Contexte & Problématique**
IaC, sécurité et limites des systèmes RAG passifs

02 **Taxonomies & Détection**
Classification des défauts et smells de sécurité

03 **Outils Spécialisés**
GLITCH, Checkov, Terrascan, Ansible-Lint

04 **LLMs pour la Sécurité IaC**
Génération sécurisée et détection sémantique

05 **LLM vs Analyseurs Statiques**
Benchmark hybride et complémentarité

06 **Architecture Agentique**
Les 6 modules du framework proposé

07 **Flux de Travail**
Validation itérative et boucle critique

08 **Apports & Perspectives**
Contributions et travaux futurs

Contexte & Problématique

Infrastructure as Code (IaC)

Outils : Terraform, Ansible, Puppet, Chef, K8s YAML

Avantages :
Automatisation, reproductibilité, scalabilité

Risques :
Security smells, mots de passe en clair, permissions excessives, dépendances obsolètes

Impact :
Pertes financières, interruptions de service, violations de données [3]

Limites du RAG Standard

- ✗ Fournit des conseils sans vérifier leur applicabilité
- ✗ Pas de validation croisée indépendante du LLM
- ✗ Correctifs non testés sur le code réel analysé
- ✗ Absence de boucle de critique itérative
- → Besoin : un agent actif, critique et vérificateur

Taxonomies des Défauts & Security Smells

2019

7 Security Smells [1]

Rahman et al. — mots de passe codés en dur, cryptographie faible… Outils : Ansible, Chef, Puppet

2020

Gang of Eight [6]

Rahman et al. — 8 catégories de défauts IaC : Configuration Data, Dependency, Security…

2021

Réplication [2]

Rahman et al. — Étude de réplication sur Ansible & Chef confirmant la taxonomie initiale

2025

62 Catégories [8]

War et al. — 7 outils IaC, alignement CWE — base de connaissances idéale pour un système RAG

→ **La taxonomie de 62 catégories (War et al., 2025) alignée CWE constitue la base de connaissances du système RAG proposé.**

Détection : des Métriques aux Modèles Sémantiques

Approche Classique

[9] *Dalla Palma et al., 2022*

Méthode

Random Forest + métriques produit/processus

Résultat

Meilleur prédicteur : métriques structurelles (tokens, entropie) > métriques de processus

Deep Learning

[4] *War et al., 2025*

Méthode

CodeBERT & LongFormer — compréhension sémantique IaC

Résultat

F1 : 0.46 → 0.92 sur Ansible (gain majeur vs méthodes classiques)

LLM + Prompt Engineering

[5] *De Vito et al., 2025*

Méthode

SecLLM — prompt engineering + cache + filtrage par score de confiance

Résultat

Surpasse GLITCH sur plusieurs datasets, gère la non-déterminisme LLM

Outils de Sécurité Spécialisés pour l'IaC

GLITCH [10]

Validateur principal

Périmètre :

Ansible, Chef, Puppet

Framework polyglotte — représentation intermédiaire pour détecter 9 smells. Précision supérieure à SLIC et SLAC.

Checkov —

Validateur Terraform/K8s

Périmètre :

Terraform, K8s

Scanner statique open-source de politiques de sécurité IaC, très utilisé en industrie.

Terrascan —

Validation croisée

Périmètre :

Multi-outils IaC

Analyse statique multi-IaC, détection de violations de conformité et de sécurité.

SecLLM [5]

Détection sémantique

Périmètre :

IaC général

Framework LLM avec caching + filtrage par score de confiance pour détecter les smells.

LLMs vs Analyseurs Statiques — Benchmark [12]

Critère	Analyseurs Statiques (SonarQube, CodeQL, SnykCode)	LLMs (GPT-4.1, Mistral Large, DeepSeek V3)
Score F1 moyen	0.26 – 0.55	0.75 – 0.80
Raisonnement contextuel	Limité (règles fixes)	Fort (compréhension sémantique)
Faux positifs	Peu	Plus nombreux
Localisation erreurs	Précise	Approximative
Déterminisme	Oui	Non (stochastique)
Bilan	**Précis mais peu de rappel**	**Meilleur rappel mais imprécis**

→ Approche hybride (LLM + analyseur statique) : combine le rappel des LLMs et la précision des outils statiques. C'est exactement ce que propose notre framework agentique.

LLMs pour la Génération IaC Sécurisée

GenSIAC [11] — Li et al., 2025

Dataset d'instruction-tuning pour rendre les LLMs conscients de la sécurité IaC.

Résultat clé : F1-score de génération sécurisée passe de

F1 = 0.30 (avant) → F1 = 0.85 (après)

Rôle dans le framework : Fine-tuning du Fix Generator pour produire des correctifs IaC sécurisés.

SecLLM [5] — De Vito et al., 2025

- Prompt engineering structuré pour la détection de smells
- Cache de résultats : réduction de la non-déterminisme LLM
- Filtrage par score de confiance : seuil adaptatif
- Surpasse GLITCH sur plusieurs benchmarks IaC
- Rôle : inspire le scoring des patches dans le Fix Generator

Architecture Agentique Proposée — 6 Modules

01 Orchestrateur Central

Gère le workflow, l'état global et prend les décisions d'itération

02 Analyseur Contextuel

CodeBERT + métriques — type IaC, zones à risque, contexte enrichi

03 Retriever RAG

Base vectorielle (62 smells + audits) — documents pertinents selon contexte

04 Fix Generator (LLM)

LLM fine-tuné (GenSIAC/SecLLM) — génère des patches + score de confiance

05 Validateur Externe

GLITCH, Checkov, Terrascan — validation indépendante du LLM

06 Formateur de Patch

Unified diff prêt à l'emploi + explication CWE + justification

Flux de Travail, Apports & Perspectives

1	2	3	4	5	6
Soumission Script IaC	Analyse Contextuelle	Retrieval RAG	Génération Patch LLM	Validation Externe	Patch Validé ✓

↺ Boucle d'itération (max 3×) si patch invalide : révision query RAG ou re-génération LLM

Apports Principaux

- Boucle de validation indépendante du LLM (GLITCH, Checkov)
- Remédiation itérative et contextuelle (au-delà de CRAG)
- Patches actionnables + diff + références CWE
- Hybridation LLM + analyseur statique (meilleur F1 combiné)

Perspectives & Travaux Futurs

- Implémentation d'un prototype & évaluation sur datasets réels
- Construction de la base vectorielle (taxonomies + rapports d'audit)
- Gestion fine de la non-déterminisme LLM (caching, seuils)
- Boucle de feedback utilisateur pour affiner les corrections